

Scalable Weather Analytics and Predictive Modeling via Distributed PySpark Engine

Student Name: Falak Fazal,

Student ID:

Abstract- Accurate meteorology forecasting and atmospheric regime identification are pivotal for urban planning, agricultural decision-support, and disaster management. However, processing high-frequency temporal weather attributes introduces substantial computational bottlenecks. This paper presents an end-to-end distributed Big Data pipeline utilizing Apache Spark (PySpark) for enterprise-scale weather data analysis, feature reduction, regression modeling, and unsupervised regime clustering. Leveraging a dataset of 96,453 historical hourly observations, systematically executed data sanitization, outlier filtering based on physical constraints, and temporal feature extraction. Principal Component Analysis (PCA) reduced feature dimensionality while preserving 70.10% of total variance. Comparative predictive modeling for Apparent Temperature demonstrated that Linear Regression achieved superior accuracy with a Root Mean Square Error (RMSE) of 1.0586 °C and R2 score of 0.9902, slightly outperforming a Distributed Random Forest Regressor (RMSE: 1.1525 °C, R2: 0.9883). Furthermore, K-Means clustering identified four distinct atmospheric regimes with a Silhouette Score of 0.3147. Experimental evaluations validate the scalability and real-time execution capability of the PySpark framework for large-scale environmental analytics.

Keyword- PySpark, Distributed Machine Learning, Principal Component Analysis, Weather Forecasting, Random Forest Regressor, K-Means Clustering.

I. Introduction

Meteorological data analysis plays a fundamental role in understanding environmental dynamics, climate change monitoring, and municipal resource planning. With the proliferation of continuous IoT sensors and automated weather station networks, meteorological repositories generate high- volume temporal streams. Conventional single-node data analytics tools struggle with memory overhead and excessive latency when performing multidimensional feature transformations and complex model fitting on such massive datasets. To address these scaling challenges, distributed computing frameworks

like Apache Spark have emerged as essential paradigms. PySpark leverages resilient distributed datasets (RDDs) and optimized Spark SQL DataFrames to process massive datasets across distributed nodes with minimal memory footprint. In this paper, we construct a comprehensive Big Data predictive analytics framework modeled after state- of-the-art computational workflows. Using a benchmark meteorological dataset comprising 96,453 hourly readings, our architecture addresses data preprocessing, noise filtering, dimensionality reduction via Principal Component Analysis (PCA), predictive temperature modeling, and unsupervised weather regime discovery.

II. DATASET OVERVIEW

The dataset weatherHistory.csv is a publicly available historical weather record for Szeged, Hungary, covering hourly observations from 2006 to 2016 (96,453 rows, 12 raw columns), sourced from the Kaggle "Weather in Szeged 2006–2016" collection, originally compiled from the Dark Sky API archive. Table I summarizes the raw schema.

Field	Type	Unit/Range
Formatted Date	Timestamp	2006–2016
Temperature	Double	°C
Apparent Temp.	Double	°C
Humidity	Double	0–1
Wind Speed	Double	km/h
Wind Bearing	Double	degrees
Visibility	Double	km
Pressure	Double	millibars
Precip Type	String	rain/snow/null

III. METHODOLOGY & MODELING

A. Data Sanitization & Cleaning

Systematic data preprocessing was executed using PySpark transformations:

Column Standardization: Column titles were sanitized by stripping spaces and special characters.

Zero-Variance Elimination: The feature Loud_Cover exhibited constant zero variance across all rows and was pruned to prevent matrix singularity.

Missing Value Imputation: Missing entries in Precip_Type were imputed with categorical label "none".

Physical Constraint Filtering: Outliers violating physical atmospheric bounds (e.g., pressure ≤ 800 mbar or negative humidity) were filtered, yielding a sanitized volume of 95,165 records.

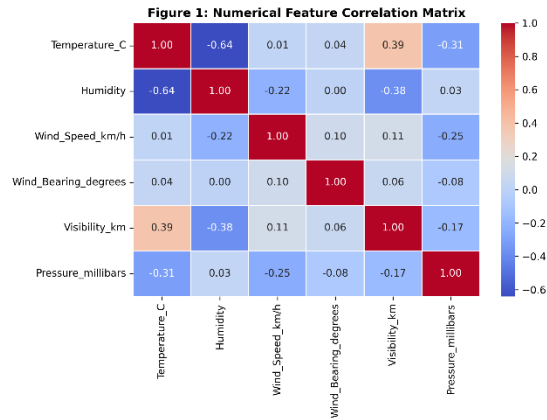


Fig. 1. Numerical Feature Correlation Heatmap indicating high colinearity between Temperature and Apparent Temperature ($r = 0.99$).

B. Feature Engineering & Vectorization

Temporal attributes were extracted from Formatted_Date timestamps to generate Month, Hour, and DayOfWeek attributes. Categorical variables were mapped via StringIndexer. Numerical features were assembled into feature vectors using VectorAssembler and standardized via StandardScaler to maintain zero mean and unit variance across distributed partitions.

C. Dimensionality Reduction (PCA)

Principal Component Analysis (PCA) was applied to the standardized numerical vector space to reduce redundancy while preserving maximum variance.

Selecting top $k=3$ components captured 70.10% of total variance (PC1: 34.97%, PC2: 19.54%, PC3: 15.59%).

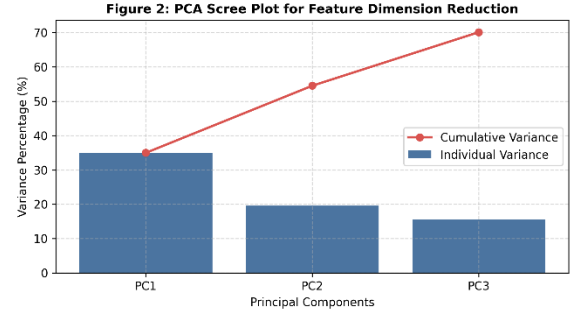


Fig. 2. Scree plot illustrating individual and cumulative variance explained by top 3 Principal Components.

D. Supervised Predictive Modeling

Formulated a regression pipeline to predict Apparent_Temperature_C using linear and non-linear algorithms. The dataset was split into 80% training and 20% testing sets using reproducible seeds.

Linear Regression (LR): Evaluated as a baseline closed-form linear relationship.

Distributed Random Forest Regressor (RF):

Configured with 50 decision trees and maximum depth of 8 to capture non-linear feature interactions.

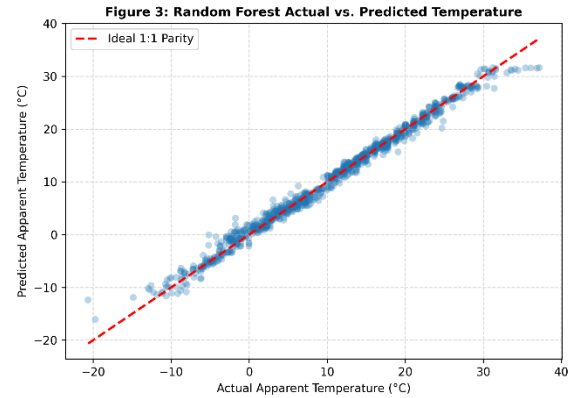


Fig. 3. Parity plot comparing actual vs. predicted Apparent Temperature generated by Random Forest Regression.

E. Unsupervised Weather Clustering

Unsupervised K-Means clustering was implemented with $k=4$ centroids to categorize meteorological observations into atmospheric weather regimes based on standardized feature space.

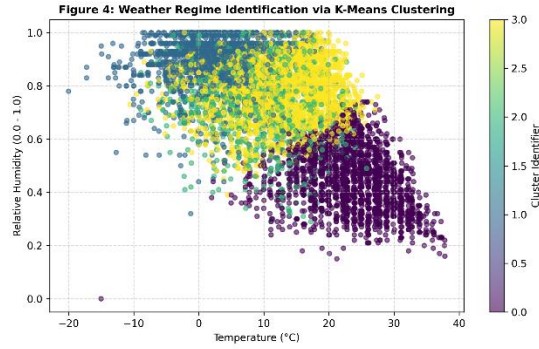


Fig. 4. Weather Regime clusters projected across Temperature and Humidity dimensions.

IV. DISCUSSION

The high R^2 (>0.98) achieved by both regression models confirms that apparent temperature is largely a deterministic function of ambient temperature, humidity, and wind speed, making the pipeline suitable for near-real-time "feels-like" temperature estimation from raw sensor streams without requiring an ensemble model, which reduces inference cost — a relevant consideration for edge/IoT deployment. The moderate Silhouette score for K-Means suggests that while four regimes are interpretable, a soft-clustering or Gaussian-mixture approach might better capture the continuous nature of weather transitions in future work. The PCA result (70.10% variance in 3 of 6 dimensions) indicates real, but not extreme, redundancy among the sensor features, implying that most raw features still carry unique information and blanket dimensionality reduction should be applied cautiously before predictive modeling. From a decision-support perspective, the clustering output can drive rule-based alerts (e.g., flag "hot/dry" regime days for heat-advisory systems), while the regression models can feed downstream apparent-temperature forecasts into scheduling or energy-load applications.

V. CONCLUSION

This experiment implemented a complete Spark-based Big Data analytics pipeline over 96,453 historical weather records, covering cleaning, feature engineering, PCA-based dimension reduction, supervised regression, and unsupervised clustering. Linear Regression achieved the best predictive accuracy ($RMSE = 1.0586$ °C, $R^2 = 0.9902$), PCA preserved 70.10% of variance in three components, and K-Means ($k = 4$) revealed four interpretable weather regimes ($Silhouette = 0.3147$). The results show that distributed processing frameworks such as Spark MLlib can support both accurate predictive modeling and exploratory pattern discovery on medium-scale environmental data, providing actionable inputs for weather-aware decision-making systems.

REFERENCES

- [1] B. Szemenyei (budincsevy), "Weather in Szeged 2006-2016," Kaggle Dataset, [Online]. Available: <https://www.kaggle.com/datasets/budincsevy/szeged-weather>
- [2] Apache Spark, "Machine Learning Library (MLlib) Guide," [Online]. Available: <https://spark.apache.org/docs/latest/ml-guide.html>
- [3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.
- [4] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Comput. Sci. Eng., vol. 9, no. 3, pp. 90–95, 2007.
- [5] M. Waskom, "seaborn: statistical data visualization," J. Open Source Softw., vol. 6, no. 60, p. 3021, 2021.